
NIC26

Combining identity and network controls in Azure

Building for visibility, traceability and security

Kristoffer Hatland · mnemonic

NIC26

Who Am I

Kristoffer Hatland

Cloud Security Architect & Team Lead, mnemonic

Leads mnemonic's Cloud & Development team.

Consulting for 10+ years



Cloud for 7+ years
almost exclusively in Azure



Security Focus
Cloud Security Architect



Networking
Classic network background

NIC26

Why combine identity and network?

2026 Verizon DBIR 31% of breaches began with vulnerability exploitation — ahead of stolen credentials

01 Credentials are stolen

Passwords · tokens · API keys

A valid identity can still be misused.

NETWORK LIMITS REACH

Private paths and segmentation constrain access.

02 Applications are exploited

Zero-days · unpatched vulnerabilities

An exploited app retains its runtime privileges.

NETWORK + IDENTITY CONTAIN

WAF virtual patching buys time; least privilege limits harm.

ASSUME ONE CONTROL WILL FAIL

Design the other to limit the blast radius.

NIC26

Source: https://www.verizon.com/business/resources/reports/dbir/

Two control planes, designed in two rooms

Identity

Entra ID

Managed identities

Azure RBAC

PIM & Conditional Access

Owned by: **IAM / platform team**

The request



Crosses both layers.

Owned by nobody.

Network

VNets & vWAN

Private endpoints & DNS

NSGs, ASGs & Azure Firewall

Service firewalls

Owned by: **Network team**

It's all private networking. So why are we still blind?

What's deployed

- Private endpoints
- VNet integration
- Managed identities
- Diagnostic settings (some)

What we can't answer

- Which workload read this secret at 02:14?
- Through which network path?
- Could it have come another way?
- Who else could make the same call?

NIC26

Identity says who. Network says where.

Identity: who?

CONTROLS

Entra ID · managed identities · RBAC · PIM

EVIDENCE

Sign-in logs · resource logs (caller OID) · AzureActivity

Network: where?

CONTROLS

VNets · private endpoints · DNS · NSG · firewall

EVIDENCE

VNet flow logs · firewall logs · resource logs (caller IP)

Managed identities can't be targeted by Conditional Access. For most workloads, the network is the only 'from where' control you have.

NIC26

Combining Security Controls

	Broad identity	Least-privilege identity
Restricted network	**Network is the only wall** Private endpoints, but Contributor everywhere and keys still enabled.	**Defence in depth** Both layers must agree. A mistake in one is caught by the other.
Open network	**Open door** Public endpoints, shared keys, broad roles.	**Identity is the only wall** Tight RBAC, reachable from anywhere. One leaked token is enough.

NIC26

Agenda

01 The baseline

02 The reference topology and the crown jewels

03 Spoke 1: PaaS microservices

04 Spoke 2: AKS

05 Spoke 3: Microsoft Foundry

06 Seeing it, and take-aways

NIC26

01

The Baseline

Public vs service endpoint vs private endpoint

Public endpoint



SOURCE THE SERVICE SEES
Public IP (NAT GW, firewall)

DESTINATION
Public IP

SCOPE
IP rules on the service firewall

Service endpoint



SOURCE THE SERVICE SEES
Subnet private IP + VNet ID

DESTINATION
Still the public IP

SCOPE
Whole service type, per subnet

Private endpoint



SOURCE THE SERVICE SEES
Private IP

DESTINATION
Private IP in your VNet

SCOPE
One resource. Public stays open until disabled

NIC26

Anatomy of a private endpoint call

1

Resolve

FQDN → privatelink CNAME → private IP from the Private DNS zone

2

Egress

Leaves via VNet integration or an injected subnet

3

Route

Routing intent / UDR via Azure Firewall - app rules SNAT, network rules

4

Enter

Hits the PE NIC. NSG/UDR only apply if network policies are on

5

Authorise

Service checks the Entra token against RBAC, logs caller OIDC + IP

Steps 3 and 4 are where designs quietly diverge: app rules hide the source, and PE subnets ignore NSGs unless you opt in.

NIC26

VNet integration is not VNet injection

VNet integration

App Service · Functions · Logic Apps



› Outbound only - inbound needs a private endpoint

› Subnet delegated to Microsoft.Web/serverFarms

› Every app in the plan shares the subnet

VNet injection

AKS · APIM Premium · Container Apps · Foundry agents



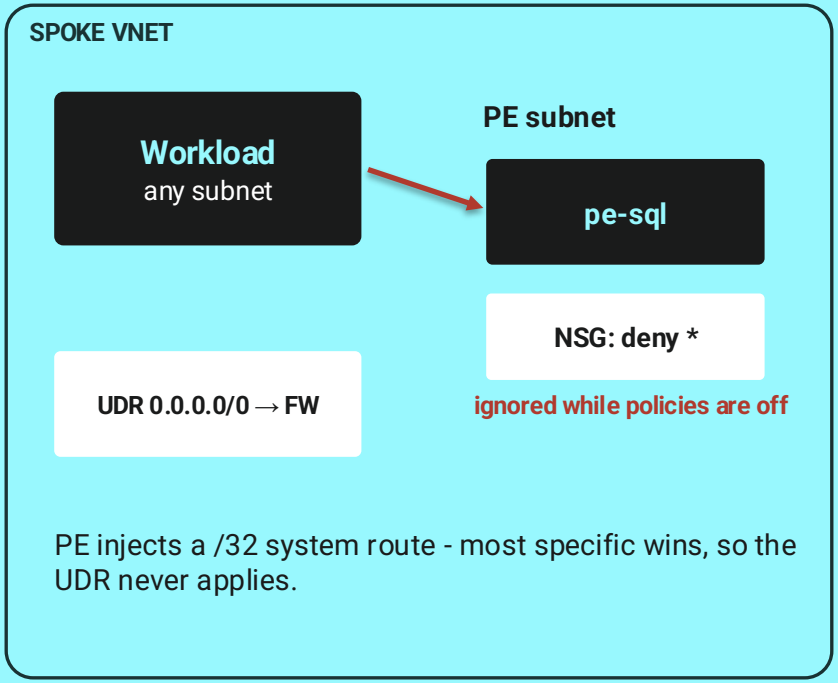
› The resource itself lives in your subnet

› Inbound and outbound through it

› NSG and UDR apply directly

NIC26

NSGs on private endpoints are opt-in



Default: off

privateEndpointNetworkPolicies is disabled on new subnets

Effect

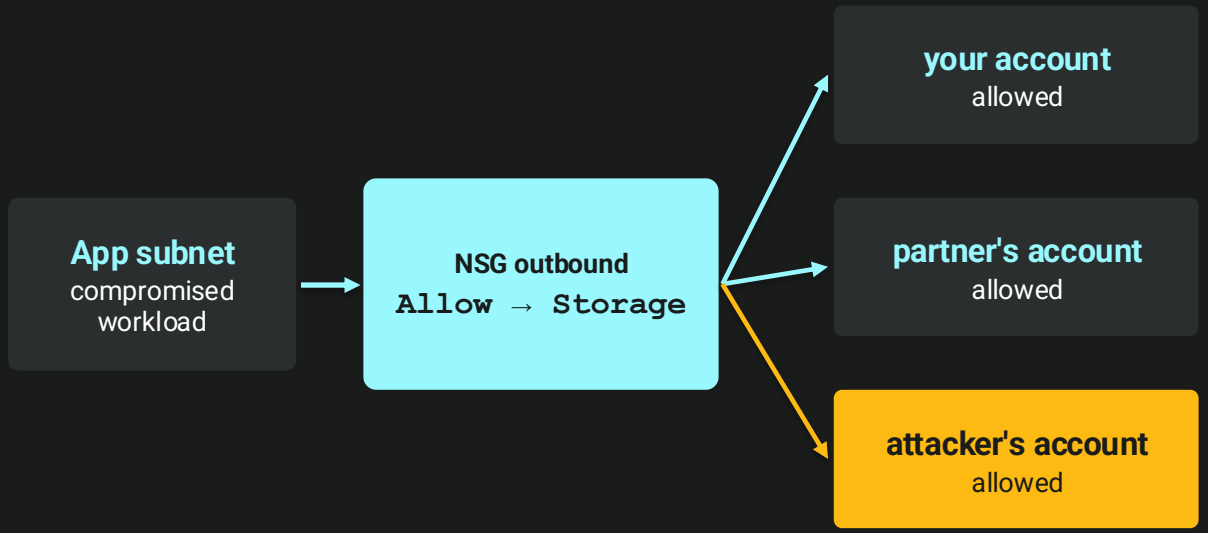
NSG rules and UDRs on the PE subnet are silently ignored

Fix

Enable NSG + route table policies on every PE subnet - enforce with policy

NIC26

Service tags: 'Storage' means every storage account



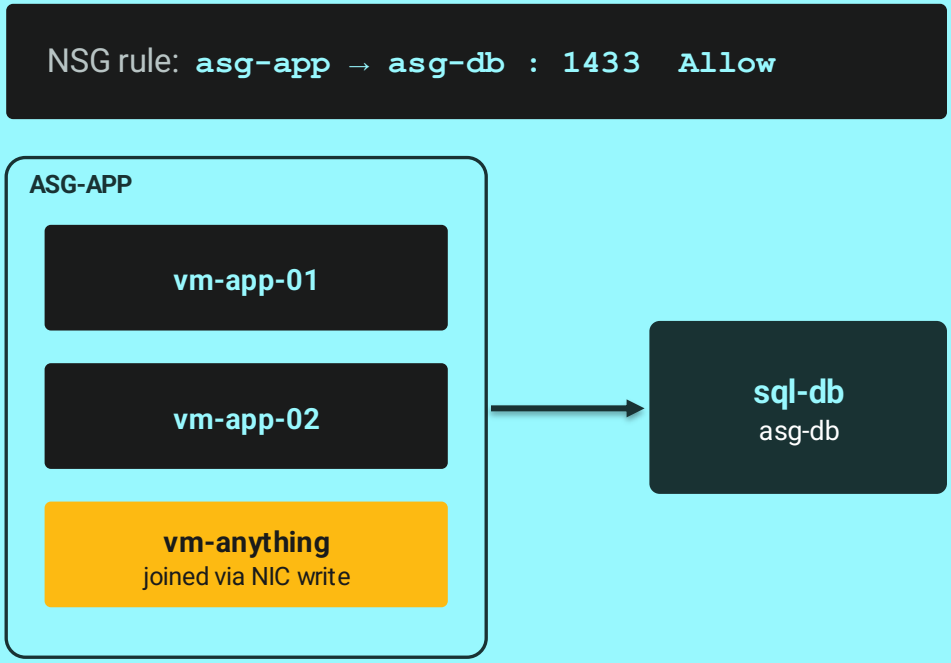
FIX

› Service endpoint policies: named storage accounts only

› Deny internet outbound - private endpoints only

› Firewall app rules with explicit FQDNs, not tags

ASGs: whoever joins the group gets the rule



Rule change
Visible in the NSG diff and review

Membership change
A NIC write - never touches the NSG

Fix
Restrict applicationSecurityGroups/joinAction and alert on NIC writes

NIC26

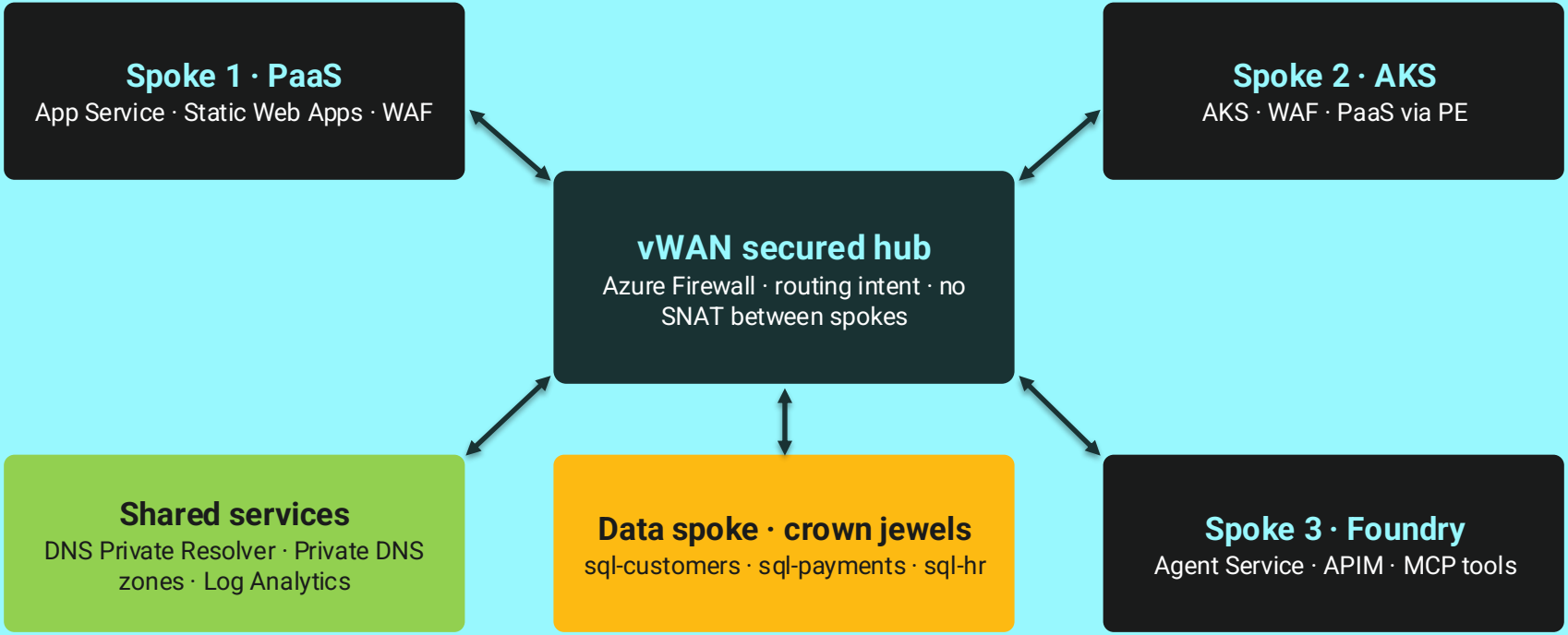
02

The reference topology

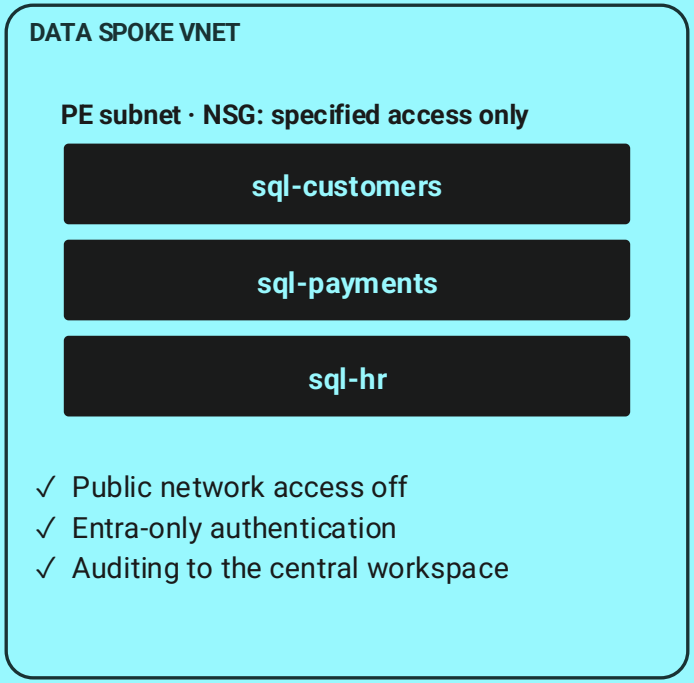
One secured hub, three workload spokes, one data spoke

NIC26

Baseline: a secured vWAN hub and its spokes



The crown jewels: a dedicated data spoke



		customers	payments	hr
api-orders	Spoke 1	✓	×	×
api-reporting	Spoke 1	×	×	×
orders pod	Spoke 2	✓	×	×
reporting pod	Spoke 2	×	×	×
orders-assistant	Spoke 3	✓	×	×
hr-agent	Spoke 3	×	×	✓

Same spoke, same path - different permissions.

NIC26

What the hub gives us - and what it can't

What it gives us

- ✓ One inspection and logging point
- ✓ Routing intent - no UDR sprawl
- ✓ DNS proxy to a central resolver
- ✓ Structured firewall logs

What it can't give us

- × Which workload made the call - only a source IP
- × The source IP, wherever app rules SNAT
- × Anything that stays inside a spoke
- × PE traffic, if network policies are off

No SNAT between spokes: keep the true caller IP

1

Network rules only

Application rules always SNAT - including MSSQL FQDN filtering and TLS inspection. Filter on IP groups and ports.

2

Prove symmetry

Microsoft recommends SNAT to private endpoints. Without it, both directions must cross the firewall: routing intent, PE network policies on - then test.

3

Know what you get

The database logs the real source IP. That's a subnet, a node or a tool - not a workload.

NIC26

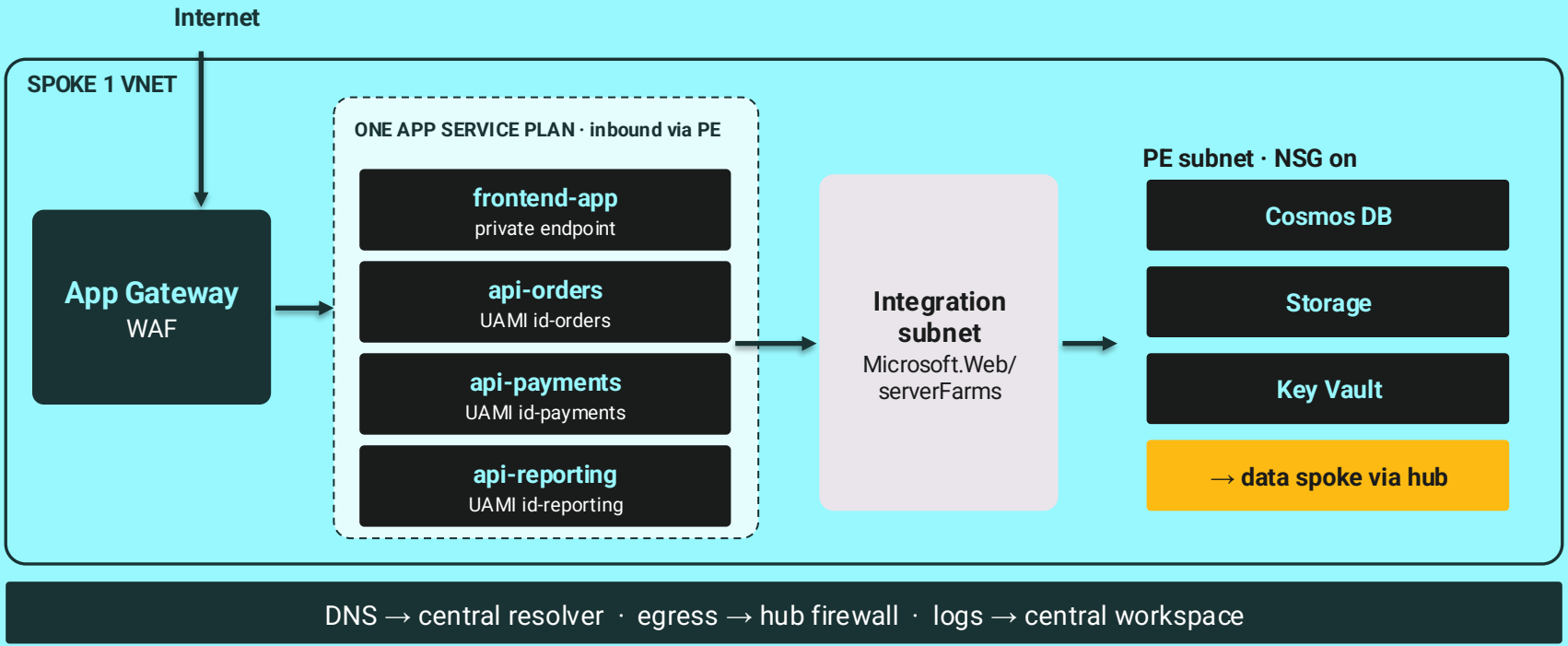
03

Spoke 1: PaaS microservices

App Service · Web Apps · PaaS data

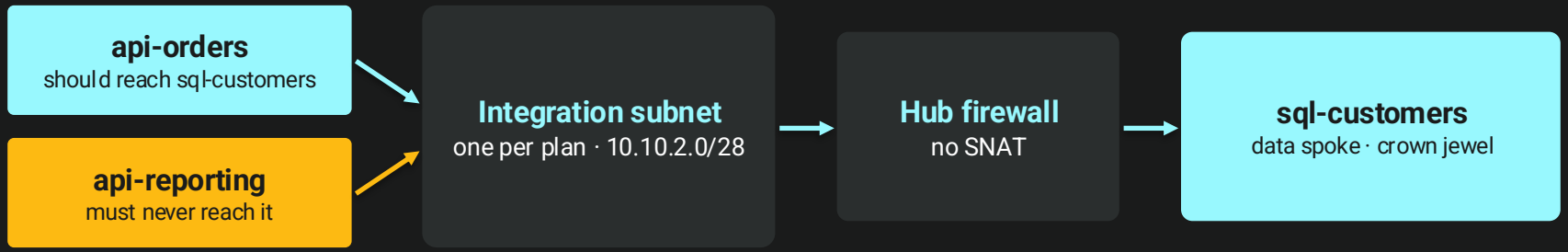
NIC26

Spoke 1: The Architecture



NIC26

api-orders may reach it. Can we prove it was api-orders?




```
AZFWNetworkRule · hub firewall
10.10.2.4 → sql-customers:1433  Allow
10.10.2.4 → sql-customers:1433  Allow
```

```
SQLSecurityAuditEvents · sql-customers
client_ip 10.10.2.4  principal id-orders
client_ip 10.10.2.4  principal id-reporting
```

The firewall allows the subnet, so both get through. Only identity can stop the wrong one.

Spoke 1: the best-practice checklist

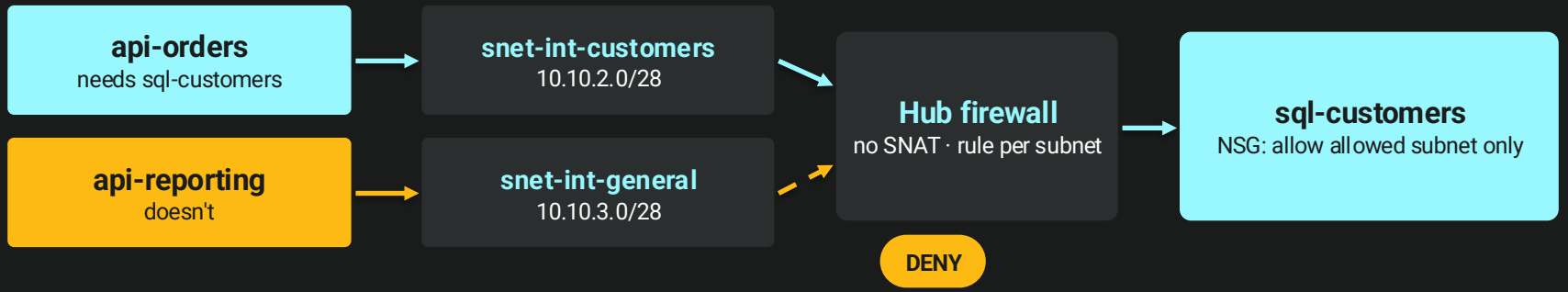
Network

- ✓ Private endpoints; disable public access
- ✓ Separate integration subnets by access profile
- ✓ Hub network rules preserve the source IP
- ✓ PE subnet policies + NSGs restrict sources
- ✓ Central DNS, WAF and firewall telemetry

Identity

- ✓ One MI per API; never share runtime IDs
- ✓ Least-privilege data roles at resource scope
- ✓ No broad control-plane roles for runtime apps
- ✓ Entra-only SQL; disable Storage Shared Key
- ✓ Audit the principal and each data operation

Split integration subnets by access profile




```
AZFWNetworkRule · hub firewall
10.10.2.4 → sql-customers:1433  Allow
10.10.3.4 → sql-customers:1433  Deny
```

```
SQLSecurityAuditEvents · sql-customers
client_ip 10.10.2.4  principal id-orders
-- nothing from id-reporting arrives --
```

Now both layers enforce it. The subnet tells the firewall and NSG who may pass; the identity confirms it.

Resolve from inside the app

```
C:\home> nslookup stordata.blob.core.windows.net

Server:    UnKnown
Address:   10.0.0.4

Non-authoritative answer:

Name:      stordata.privatelink.blob.core.windows.net
Address:   10.2.5.6

Aliases:   stordata.blob.core.windows.net
```

Find the call - and the caller

Run | Last 30 minutes | Save v | Share v | ...

```
1  AzureDiagnostics
2  | where TimeGenerated > ago(30m)
3  | where Category == "SQLSecurityAuditEvents"
4  | where database_name_s == "sql-customers"
5  | where client_ip_s == "10.10.2.4"
6  | where server_principal_name_s in ("id-orders", "id-reporting")
7  | project event_time_t, client_ip_s, server_principal_name_s,
8            action_name_s, succeeded_s
```

Results | Chart

event_time_t	client_ip_s	server_principal_name_s	action_name_s	succeeded_s
2026-10-11 16:33:21	10.10.2.4	id-reporting	SELECT	0
2026-10-11 16:32:05	10.10.2.4	id-orders	SELECT	1

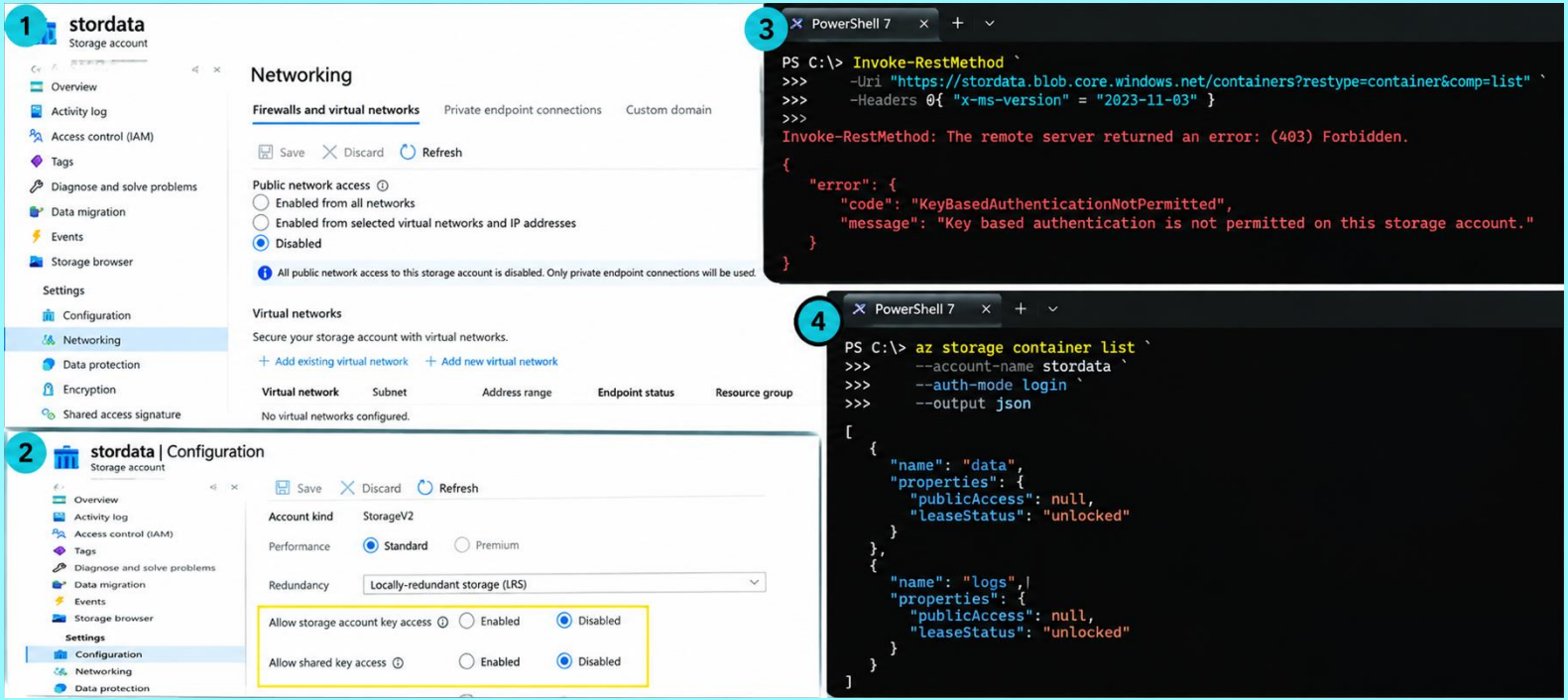
Run | Last 30 minutes | Save v | Share v | ...

```
1  AZFWNetworkRule
2  | where TimeGenerated > ago(30m)
3  | where SourceIp == "10.10.2.4" and DestinationIp == "10.2.5.4"
4  | where DestinationPort == 1433
5  | project TimeGenerated, SourceIp, DestinationIp, DestinationPort,
6            Action, Rule
7  | order by TimeGenerated desc
```

Results | Chart

TimeGenerated	SourceIp	DestinationIp	DestinationPort	Action	Rule
2026-10-11 16:33:21	10.10.2.4	10.2.5.4	1433	Allow	allow-appsvc-sql
2026-10-11 16:32:05	10.10.2.4	10.2.5.4	1433	Allow	allow-appsvc-sql

Close the side door



NIC26

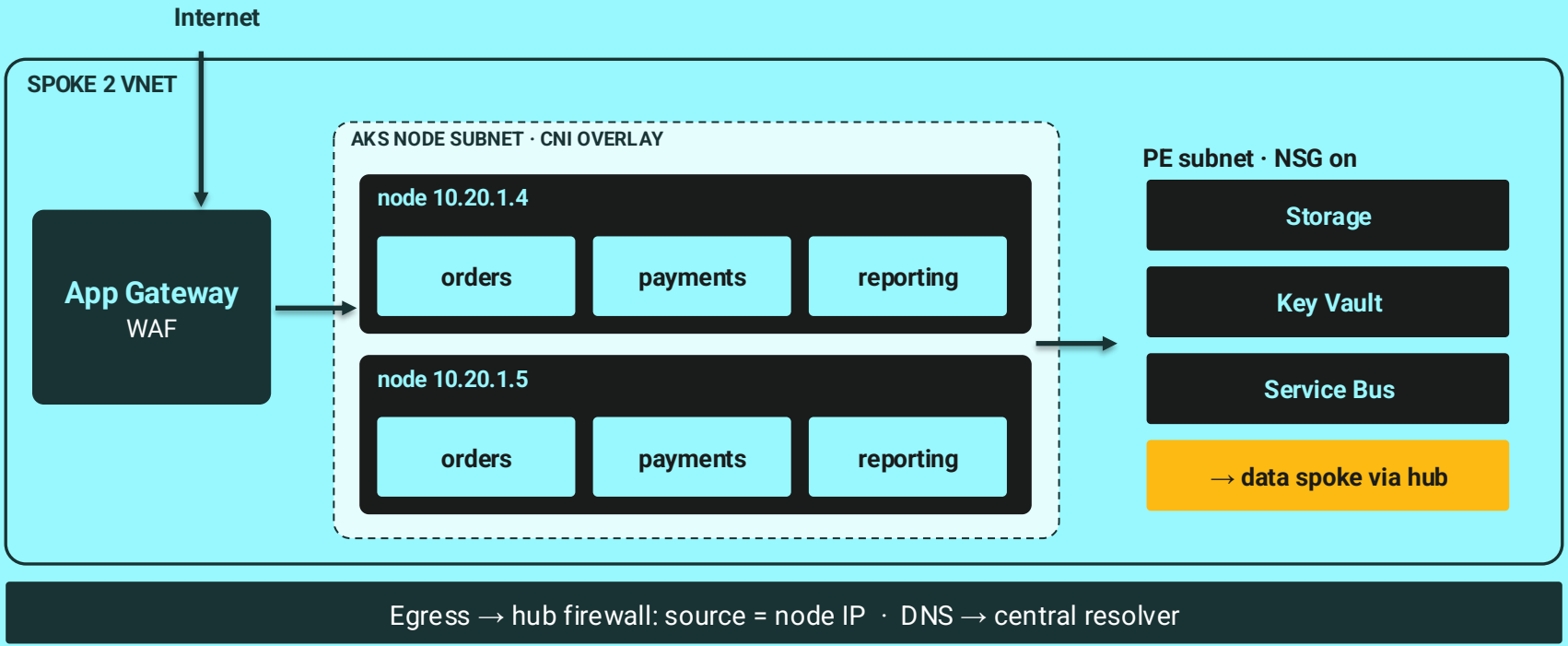
04

Spoke 2: AKS

Where the network loses the workload

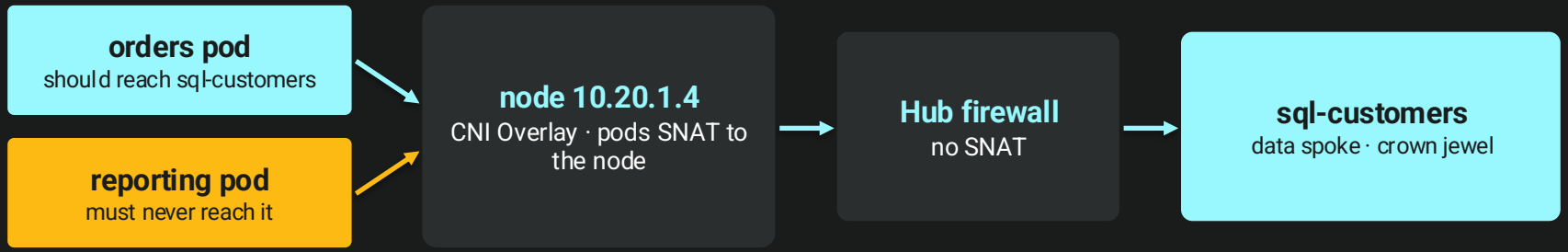
NIC26

Spoke 2: the architecture



NIC26

The firewall sees a node, not a workload




```
AZFWNetworkRule · hub firewall
10.20.1.4 → sql-customers:1433  Allow
10.20.1.4 → sql-customers:1433  Allow
```

```
SQLSecurityAuditEvents · sql-customers
client_ip 10.20.1.4  principal id-orders
client_ip 10.20.1.4  principal id-reporting
```

Every pod on a node looks like one IP. Without Workload ID, they'd look like one identity too.

Spoke 2: the best-practice checklist

Network

✓ Separate sensitive workloads by node pool/subnet

✓ Enforce placement with taints and selectors

✓ Restrict SQL access to approved node subnets

✓ Enable PE subnet policies; check return routing

✓ ACNS/Cilium flows show pod and verdict

Identity

✓ Entra Workload ID per service account

✓ Federate each account to its own UAMI

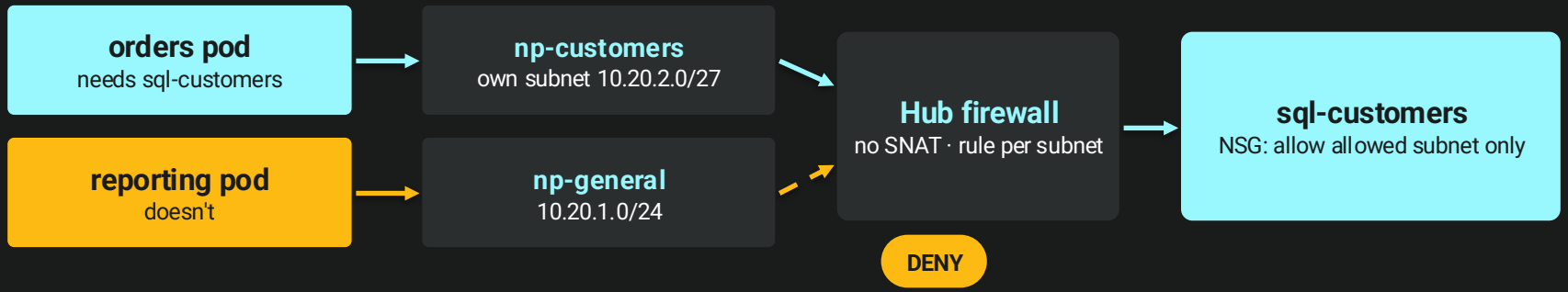
✓ Least-privilege SQL database permissions

✓ No SQL passwords or shared keys in pods

✓ Correlate SQL identity with pod flow logs

NIC26

Dedicated nodes and egress IPs for dedicated workloads




```
AZFWNetworkRule · hub firewall
10.20.2.4 → sql-customers:1433  Allow
10.20.1.4 → sql-customers:1433  Deny
```

```
SQLSecurityAuditEvents · sql-customers
client_ip 10.20.2.4  principal id-orders
-- nothing from id-reporting arrives --
```

The source IP means something again. Workload ID still names the pod - two walls, not one.

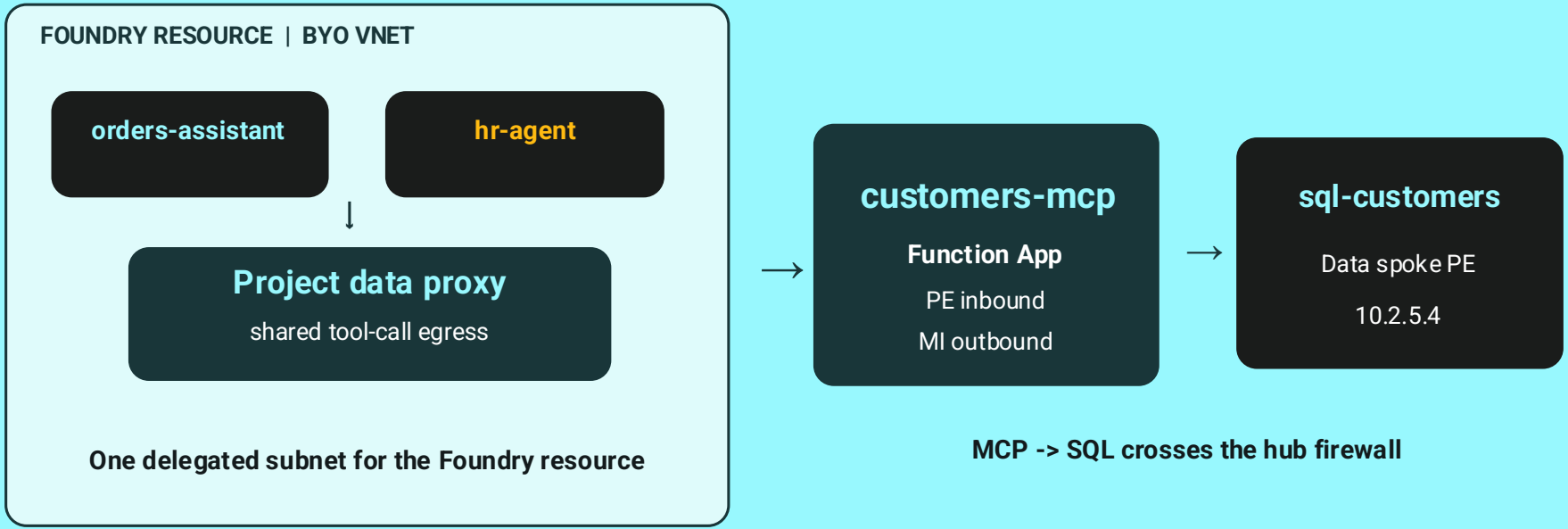
05

Spoke 3: Microsoft Foundry

Agents, AI gateway and MCP tools

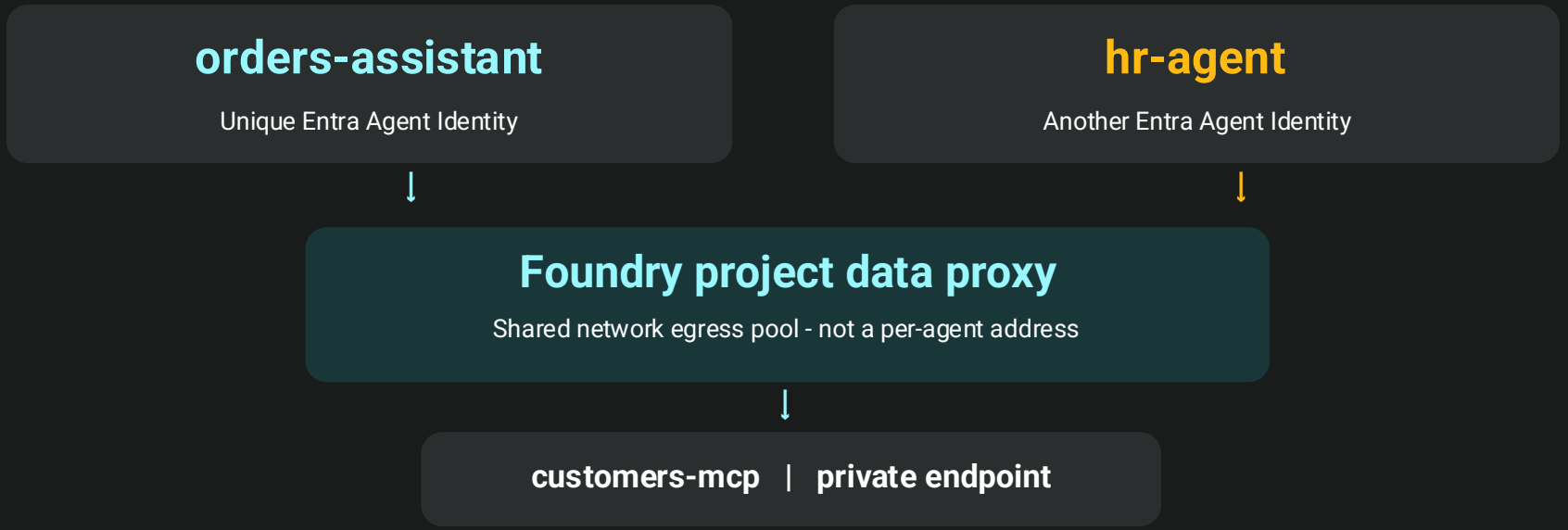
NIC26

Spoke 3: The Initial Path



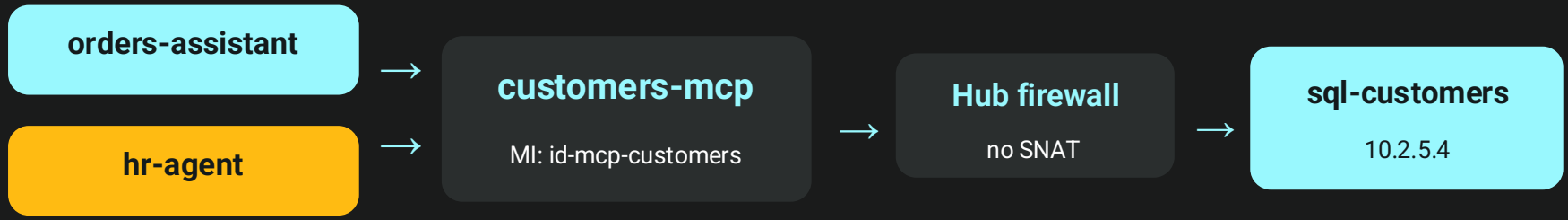
BYO Storage, Cosmos DB and AI Search use their own private endpoints

Two agent identities. One network path.



The PE can see the project egress - not which agent asked.

The database sees the tool, not the agent



AZFWNetworkRule

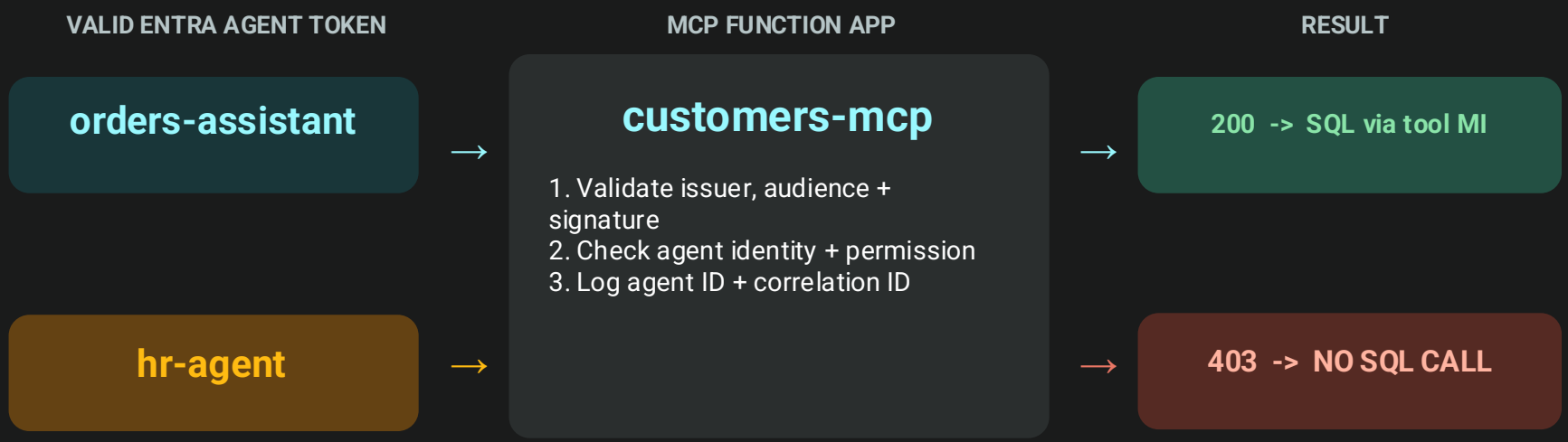
```
10.30.3.4 -> 10.2.5.4:1433  Allow
10.30.3.4 -> 10.2.5.4:1433  Allow
```

SQL audit - sql-customers

```
principal = id-mcp-customers
principal = id-mcp-customers
```

If both callers reach the tool, SQL cannot reconstruct which agent initiated it.

The MCP must authorize the agent

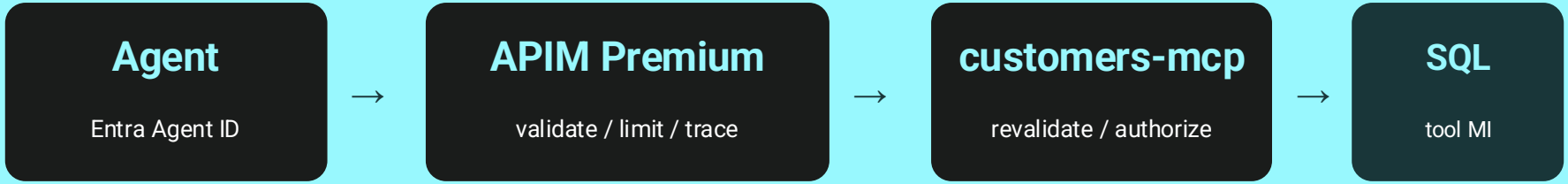


API/app-role authorization is enforced by the MCP service; Function resource IAM alone is not HTTP authorization.

Tool log = agent attribution. SQL audit = tool identity.

Now introduce APIM: policy and evidence

Route the Foundry MCP connection to APIM explicitly



MCP route

Forward the validated agent JWT unchanged to the MCP; the tool checks operation permissions.

Model route (separate)

APIM may use its own managed identity to configured Foundry model backends.

APIM adds central controls. It does not replace agent or tool identity.

Spoke 3: the best-practice checklist

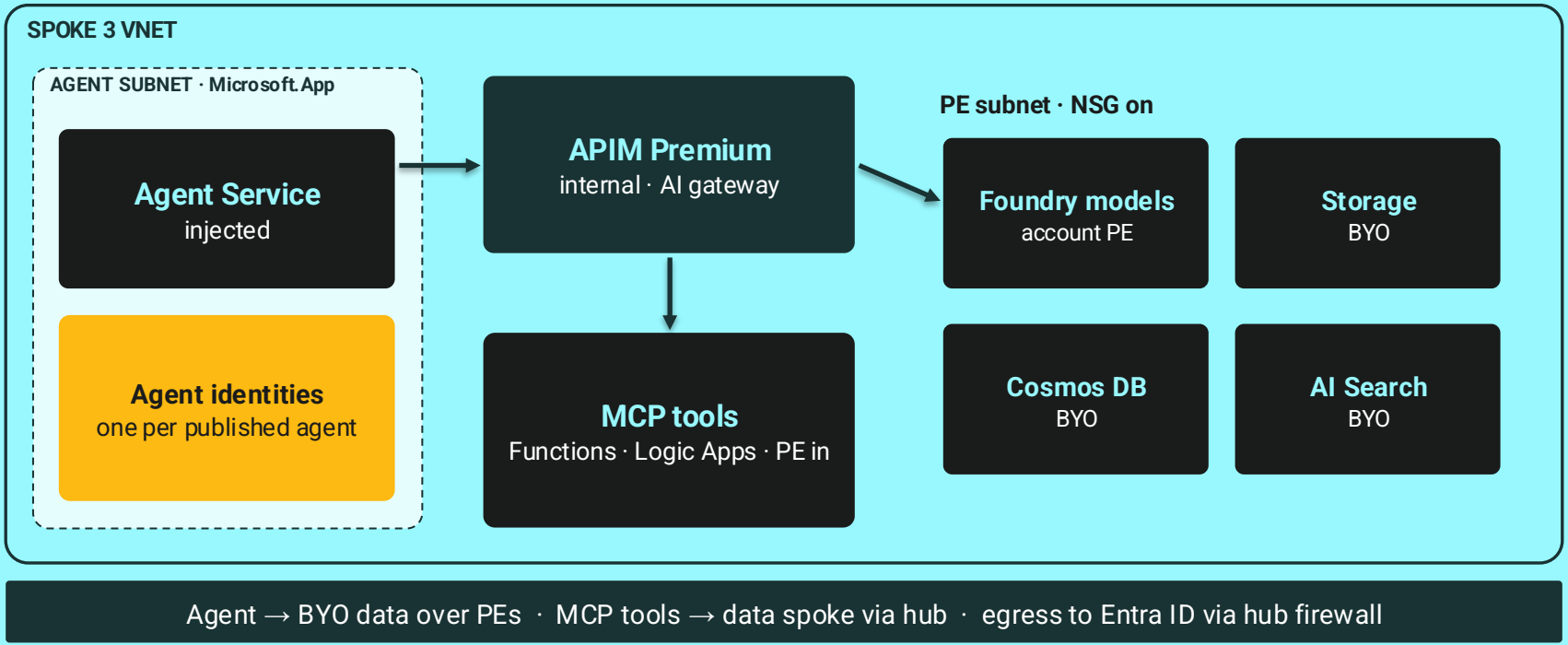
Network

- ✓ Network-secured Foundry; size delegated subnet
- ✓ Shared project proxy: not a per-agent IP
- ✓ Separate Foundry resources by trust domain
- ✓ Private PEs for MCP, Foundry and BYO data
- ✓ PE NSG policies; approved sources only
- ✓ Tool -> SQL via hub FW; optional internal APIM

Identity

- ✓ New-model Agent ID is unique at creation
- ✓ MCP connection uses agentic-identity token
- ✓ APIM validates/forwards; MCP revalidates
- ✓ MCP authorizes operation per agent identity
- ✓ Tool MI has least-privilege SQL access
- ✓ Log agent ID, request ID and tool outcome

Spoke 3: the architecture



Agent → BYO data over PEs · MCP tools → data spoke via hub · egress to Entra ID via hub firewall

NIC26

06

Seeing it, and take-aways

Three spokes, one pattern

	THE NETWORK CAN TELL YOU	IDENTITY MUST TELL YOU
Spoke 1 · PaaS	the access profile - if integration subnets are split	the app - one UAMI per app
Spoke 2 · AKS	the access profile - with dedicated nodes / egress IPs	the workload - Workload ID per service account
Spoke 3 · Foundry	project-proxy egress; then the tool IP	the agent in API/tool traces; SQL sees the tool

Design the network to name the access profile. Identity names the workload.

NIC26

Five questions, five log sources

	SOURCE	JOIN ON
Who called?	Resource logs: StorageBlobLogs, Key Vault AuditEvent	**Caller object ID · auth type**
Which identity?	Managed identity, service principal and agent sign-in logs	**Service principal ID**
Which pod or agent?	ContainerNetworkLogs · APIM gateway logs	**Pod / caller · time**
From where, which path?	VNet flow logs, Azure Firewall logs	**Source / dest IP · SNAT**
Who changed the rules?	AzureActivity	**Role, NIC, NSG changes**

NIC26

Join identity and network in one query

```
StorageBlobLogs
| where TimeGenerated > ago(24h)
| summarize Calls = count(), Ops = make_set(OperationName)
    by AccountName, AuthenticationType,
       RequesterObjectId, CallerIpAddress
| join kind=leftouter (
    AADManagedIdentitySignInLogs
    | distinct ServicePrincipalId, ServicePrincipalName
  ) on $left.RequesterObjectId == $right.ServicePrincipalId
| project AccountName, AuthenticationType,
    Identity = ServicePrincipalName,
    CallerIpAddress, Calls, Ops
```

`Identity`
which workload - if identities are per workload

`CallerIpAddress`
the true source: a subnet or a node - not a workload

`AuthenticationType ≠ OAuth`
a side door: key or SAS, no identity

Make it stick: guardrails as code

Deny

Public network access on PaaS services

Deny

Shared key and local authentication

DeployIfNotExists

Diagnostic settings → central workspace

DeployIfNotExists

PE network policies + Private DNS records

+ Audit: MI role assignments above resource-group scope · NIC writes that join ASGs

NIC26

Take-aways

1. **Design per flow, not per layer** - every flow gets a path and an identity
2. **Treat the baseline as security controls** - PE network policies, service tags, ASGs
3. **Make the source IP mean something** - subnets and egress IPs per access profile
4. **Let identity carry the rest** - one identity per workload, agent and tool
5. **Make logs joinable** - object ID, IP and resource in one workspace
6. **Enforce with policy** - so the design survives the next deployment

NIC26

Thank you



Questions?

Kristoffer Hatland

NIC26